

Five Structural Theorems and Three Conjectures About Automated Theorem Provers

With an Analysis of P vs. NP and Lower Bounds on Shortest Proof Length

Prepared for Brian Tenneson

July 30, 2026

Abstract

This document typesets and lightly edits the immediately preceding analysis of automated theorem provers (ATPs) in the semi-ideal computer framework. It presents five structural theorems, three open conjectures, a conditional analysis of whether ATPs can settle P versus NP, and a hierarchy of possible lower bounds on the shortest proof length of $P \neq NP$. The framework and terminology are drawn primarily from Brian Tenneson's uploaded works, especially *Depths of a Simulation and Formalized Self-Awareness*, *Length, Not Height*, *Predator 7.1 Documentation* and *Cross-Branch Reuse Rate Theory*, and *ML-SIC-ATP Theory*.

Contents

1	Scope and notation	3
2	Five structural theorems about ATPs	3
2.1	The Birth-Ledger Normal-Form Theorem	3
2.2	The Chronicle-Lattice Theorem	4
2.3	The Finite-Universe Novelty-Budget Theorem	4
2.4	The No-Free-Doppelganger Theorem	5
2.5	The Certified-Completion Impossibility Theorem	5
3	Three conjectures not immediately settled	6
3.1	The Losing-Branch Compiler Conjecture	6
3.2	The Settlement-Region Realization Conjecture	6
3.3	The Admissible Learning Separation Conjecture	6
4	Can ATPs solve P versus NP?	7
4.1	Maybe yes in reality	7
4.2	Definitely yes, conditionally	7
4.3	Definitely not guaranteed	8
4.4	Why arithmetical tier is not the decisive issue	8
5	Lower bounds on the shortest proof of $P \neq NP$	8
5.1	The bound must be system-relative	8
5.2	Elementary floors	9
5.3	A potential-increment lower-bound scheme	9

5.4	The definitional floor	10
5.5	The indispensable-dependency floor	10
5.6	Restricted proof-system lower bounds	11
5.7	Why the standard barriers do not yield line bounds	11
5.8	What can honestly be claimed	11
6	Conclusion	12
	Source note	12

1 Scope and notation

Fix a formal system

$$F = (x, y, z),$$

a formula domain $Y \subseteq y$, and an input set of hypotheses $\Gamma \subseteq Y$.

In the framework used here, a semi-ideal computer over Y is a pair

$$M = (C, H)$$

with a stage-state map

$$C : \mathcal{P}(Y) \times \mathbb{N}^+ \longrightarrow \mathcal{P}(Y)$$

and a halting map

$$H : \mathcal{P}(Y) \times \mathbb{N}^+ \longrightarrow \{0, 1\}.$$

Its state begins at the input, grows monotonically, has a persistent halting signal, and freezes after halting. An automated theorem prover is a SIC intended either to enumerate consequences or to search for a designated target. An enumerating ATP is deductively effective; a target-search ATP for φ must halt only after φ is present and must eventually halt whenever $\Gamma \vdash_F \varphi$.

For a sentence L , define the shortest proof length relative to F and Γ by

$$\|L\|_{F,\Gamma} = \min\{|\pi| : \pi \text{ is an } F\text{-proof of } L \text{ from } \Gamma\},$$

with value ∞ when no such proof exists.

The definition of ATP is deliberately broad. Therefore, the strongest statements true of every ATP are structural statements about monotone histories, halting, recoding, and computability. Claims about sound inference additionally require hypotheses such as rule-drivenness, deductive effectiveness, or computability.

2 Five structural theorems about ATPs

2.1 The Birth-Ledger Normal-Form Theorem

Theorem 2.1 (Birth-Ledger Normal Form). *Let $M = (C, H)$ be any ATP and fix an input Γ . Define the birth time of a formula $\psi \in Y$ by*

$$b_{M,\Gamma}(\psi) = \min\{m \in \mathbb{N}^+ : \psi \in C(\Gamma, m)\},$$

with $b_{M,\Gamma}(\psi) = \infty$ if ψ never appears. Then, for every stage m ,

$$C(\Gamma, m) = \{\psi \in Y : b_{M,\Gamma}(\psi) \leq m\}.$$

If h is the first halting stage, no formula has a finite birth time greater than h .

Proof. Monotonicity makes membership irreversible. Thus every formula that ever appears has a unique least stage at which it appears, and it remains present at every later stage. This proves the displayed representation of $C(\Gamma, m)$.

If $H(\Gamma, h) = 1$, the freezing axiom gives

$$C(\Gamma, h) = C(\Gamma, h+1) = C(\Gamma, h+2) = \dots$$

Therefore no new formula can first appear after h . □

Conversely, a birth-time assignment satisfying the initial-state condition and admitting no new births after a chosen finite halting time determines an abstract ATP run. Thus an ATP computation can be viewed as

a ledger of formula birthdays together with an optional event horizon.

This normal form separates two kinds of information that are often conflated: the order in which formulas become available and the independent halting observation attached to the run.

2.2 The Chronicle-Lattice Theorem

Theorem 2.2 (Chronicle Lattice). *For any ATP M , fixed input Γ , and stages $m, n \in \mathbb{N}^+$,*

$$C(\Gamma, m) \cap C(\Gamma, n) = C(\Gamma, \min\{m, n\}),$$

and

$$C(\Gamma, m) \cup C(\Gamma, n) = C(\Gamma, \max\{m, n\}).$$

Hence the stage states of a single run form a linearly ordered distributive lattice under inclusion.

Proof. Assume without loss of generality that $m \leq n$. Monotonicity gives

$$C(\Gamma, m) \subseteq C(\Gamma, n).$$

The intersection is therefore $C(\Gamma, m)$, while the union is $C(\Gamma, n)$. □

The theorem draws a clean distinction between a branching *search space* and a completed *run*. The ambient theorem graph may have many incomparable branches, but the history of one run contains no incomparable memories: it is always a chain.

2.3 The Finite-Universe Novelty-Budget Theorem

Theorem 2.3 (Finite-Universe Novelty Budget). *Suppose an ATP operates over a finite formula domain Y . On any input $\Gamma \subseteq Y$, its state sequence has at most*

$$|Y \setminus \Gamma|$$

strict transitions. Consequently, its state eventually becomes constant, whether or not the ATP ever halts.

Proof. Each strict transition

$$C(\Gamma, m) \subsetneq C(\Gamma, m+1)$$

introduces at least one member of Y not previously present. At stage one, the state is Γ , so only the $|Y \setminus \Gamma|$ remaining formulas can ever be new. Hence there can be at most that many strict transitions. □

This yields the important non-implication

$$\text{eventual state stabilization} \not\Rightarrow \text{halting}.$$

An ATP may stop producing new formulas while its halting bit remains 0 forever. It may also remain in the same state for many stages before changing its halting bit. Stabilization is a fact about the theorem state; halting is an additional observation.

2.4 The No-Free-Doppelganger Theorem

Theorem 2.4 (No-Free Doppelganger). *Let $t : Y \rightarrow Y^t$ be an invertible syntactic recoding, and let M^t be the recoded copy of an ATP M . Then, for every input Γ and stage m ,*

$$C^t(t[\Gamma], m) = t[C(\Gamma, m)].$$

Consequently,

$$b_{M^t, t[\Gamma]}(t(\psi)) = b_{M, \Gamma}(\psi),$$

first halting stages agree, strict-change stages agree, and all corresponding transaction counts agree exactly, including the value ∞ .

Proof. The recoded machine is defined by pulling a state back through t^{-1} , applying the original transition and halting maps, and then transporting the resulting state forward through t . Since t is bijective, this operation preserves membership, inclusion, first appearance, and the halting observation stage by stage. $\square$

Thus changing notation, renaming formulas, altering Goedel numbers, or applying an invertible translation cannot by itself accelerate proof search. A recoded ATP is a perfect syntactic doppelganger, not a stronger or faster prover. Real speedup must come from some structural change: a different rule set, compiled lemmas, derived rules, a different policy, additional side information, or a different proof system.

2.5 The Certified-Completion Impossibility Theorem

Theorem 2.5 (Certified Completion Is Impossible for Undecidable Consequence). *Suppose the consequence problem for F on Y is undecidable. Then there is no computable ATP having all three of the following properties on every input Γ :*

1. *it eventually reaches the complete consequence set*

$$C(\Gamma, m) = \text{Con}_F(\Gamma) \cap Y;$$

2. *it eventually halts;*

3. *its halt certifies that no additional consequences remain.*

Proof. Given an instance (Γ, ψ) of the consequence problem, run the ATP until its certified completion stage m . Then decide whether

$$\psi \in C(\Gamma, m).$$

By the assumed completion guarantee, this test is equivalent to deciding whether $\Gamma \vdash_F \psi$. That contradicts the undecidability of the consequence problem. $\square$

The theorem supports a useful slogan:

A computable ATP may enumerate an undecidable theory forever, but it cannot always know when it has finished.

3 Three conjectures not immediately settled

3.1 The Losing-Branch Compiler Conjecture

Conjecture 3.1 (Losing-Branch Compiler). *There exists a computably axiomatized formal system F and a natural target family (c_n) such that a settlement search sharing lemmas between the c_n and $\neg c_n$ branches has an unbounded advantage over two independent proof-covering searches:*

$$\frac{\sigma_{\text{ind}}(c_n)}{\sigma_{\otimes}(c_n)} \longrightarrow \infty,$$

and no fixed finite collection of precompiled derived rules reproduces that advantage for every n .

Existing cross-branch reuse theory shows that the branch pursuing the ultimately false side may generate lemmas useful to the branch that eventually halts. Such insertion can be interpreted as conservative compilation and can yield a nonnegative settlement dividend. The unresolved part is to exhibit a natural infinite family with a genuine asymptotic separation that cannot be simulated by one fixed finite macro-library.

The obstacle is familiar: proving the unbounded separation seems to require a lower bound on all relevant proofs in the unshared or uncompiled system. That is precisely the kind of strong proof-length lower bound that present proof-complexity techniques rarely provide.

3.2 The Settlement-Region Realization Conjecture

Conjecture 3.2 (Settlement-Region Realization). *Restrict the clocked-simulation preorder to effective, target-preserving simulations. Every effectively presented principal up-set*

$$\uparrow M = \{N : M \preceq N\}$$

is the exact settlement region

$$U_c = \{N : N \text{ settles } c\}$$

of some formalized conjecture c .

A negative answer would reveal previously unknown restrictions on which up-sets can arise as settlement regions. A positive answer would say that formalized conjectures can carve out essentially arbitrary principal regions of effective ATP space.

The difficulty is that settlement is not merely a property of simulation strength. It also depends on the target language, the interpretation of the target and its negation, soundness assumptions, and whether the simulation preserves the relevant observations. A realization theorem would have to encode all of these constraints without accidentally enlarging or shrinking the desired region.

3.3 The Admissible Learning Separation Conjecture

Conjecture 3.3 (Admissible Learning Separation). *There exists a natural infinite theorem family and a computable heuristic h , learned from earlier proof data, such that:*

1. *h is admissible: it never overestimates remaining proof distance;*
2. *A^* guided by h always returns a shortest proof;*

3. *the guided search materializes superpolynomially fewer proof states than breadth-first search;*
4. *both searches use exactly the same inference rules and lemma library.*

The known boundary results are sharp but do not settle this conjecture. A learned policy cannot produce a target below its closure depth. A proof-covering wrapper can preserve completeness. An admissible heuristic preserves shortest-path optimality. What remains unclear is whether a learned heuristic can be simultaneously admissible, substantially informative, computationally affordable, and transferable across an infinite natural family.

4 Can ATPs solve P versus NP?

The current official status is that P versus NP remains unsolved. The Clay Mathematics Institute continues to list it as a Millennium Prize Problem.¹

The correct answer is not a simple yes or no. It is a set of conditional statements.

4.1 Maybe yes in reality

An ATP could solve P versus NP by producing a formally valid proof of either

$$P = NP \quad \text{or} \quad P \neq NP.$$

No known theorem excludes a machine-discovered proof. If a finite proof exists in a recursively axiomatized formal system F , then a fair unbounded enumeration of all F -proofs eventually finds it. A settlement machine dovetailing the two searches eventually finds whichever side is provable.

This is only an existence guarantee. It says nothing about whether the proof fits in available memory, can be generated before the physical universe ends, or can be found by a practical search policy.

4.2 Definitely yes, conditionally

If

$$F \vdash P \neq NP$$

and M is a complete fair ATP for F , then M eventually produces a proof. For the canonical breadth-first target-search ATP,

$$\tau_{B_F}(P \neq NP) = \|P \neq NP\|_{F,\Gamma}.$$

This is the proof-horizon principle: the canonical breadth-first target machine halts exactly at the length of a shortest proof.

The conditional statement is mathematically strong but epistemically limited. Neither the provability of $P \neq NP$ in a preferred strong theory nor the size of its shortest proof is presently known.

¹Clay Mathematics Institute, “P vs NP,” <https://www.claymath.org/millennium/p-vs-np/>, accessed July 2026.

4.3 Definitely not guaranteed

An ATP restricted to a fixed formal system F is not guaranteed to settle the problem. Failure may occur for several distinct reasons:

- neither $P \neq NP$ nor $P = NP$ is provable in F ;
- the ATP's implementation omits necessary inference rules;
- its language lacks a usable definitional development of machines, time bounds, and complexity classes;
- its search policy permanently prunes every successful proof;
- a proof exists but is physically or computationally inaccessible;
- the statement is independent of the selected axiom system.

The Ascent experiments illustrate the rule-set obstruction particularly clearly. The original system lacked rules introducing negation, reductio, and related constructions. Since $P \neq NP$ is a negation, increasing search-depth or quantifier-tier parameters could not make it derivable. Search parameters bound the exploration of a calculus; they do not enlarge the calculus.

4.4 Why arithmetical tier is not the decisive issue

P versus NP can be arithmetized and placed at low finite height in the arithmetical hierarchy. But tier classifies the form of the quantifier prefix, not the shortest proof length, not reducibility, and not practical search difficulty. A prover may handle many Π_2^0 sentences while remaining nowhere near $P \neq NP$.

Thus the best verdict is:

ATPs are not known either to be incapable of solving P versus NP or destined to solve it. A fair ATP will find a proof if its formal system contains one, but existence, formal-system provability, manageable length, and feasible discovery are all currently unknown.

5 Lower bounds on the shortest proof of $P \neq NP$

5.1 The bound must be system-relative

There is no system-independent value of

$$\|P \neq NP\|.$$

The meaningful quantity is

$$\|P \neq NP\|_{F,\Gamma}.$$

Adding definitions, extension variables, lemmas, cut, derived rules, or $P \neq NP$ itself as an axiom can change the value radically. Any numerical claim must therefore specify:

- the formal system;
- the hypothesis set;

- whether theorem citations count as one step or are fully expanded;
- whether length means raw proof-file steps, logical lines, or total symbol size.

The uploaded Predator analysis distinguishes raw Metamath steps from logical inference lines. In a random sample of 6,000 proofs, the median raw length was 173 while the median logical length was 12, and 95.3 percent of executed steps were formula construction rather than inference. Therefore a claim about “proof length” is incomplete until its unit is fixed.

5.2 Elementary floors

The weakest rigorous lower bounds are immediate.

If $P \neq NP$ is neither a hypothesis nor an axiom, then under ordinary line-count conventions,

$$\|P \neq NP\|_{F,\Gamma} \geq 2.$$

For total symbol size,

$$\|P \neq NP\|_{F,\Gamma}^{\text{size}} \geq |\ulcorner P \neq NP \urcorner|,$$

because the final displayed formula must itself occur in the proof.

These bounds are formally correct and practically negligible.

5.3 A potential-increment lower-bound scheme

A more imaginative general method is to construct a rank or potential

$$r : Y \longrightarrow \mathbb{N}$$

that every inference rule can increase only slowly.

Suppose every direct inference from premises $\alpha_1, \dots, \alpha_k$ to conclusion β satisfies

$$r(\beta) \leq \max_i r(\alpha_i) + d$$

for a fixed $d > 0$. Let

$$r_0 = \sup_{\gamma \in \Gamma} r(\gamma).$$

Then every proof of a target L satisfies

$$\boxed{\|L\|_{F,\Gamma} \geq 1 + \left\lceil \frac{r(L) - r_0}{d} \right\rceil}.$$

Indeed, after $n - 1$ inference steps, the rank can have risen by at most $(n - 1)d$ above the initial maximum. Reaching rank $r(L)$ therefore requires the displayed number of lines.

If $d = 0$ and $r(L) > r_0$, then L is unprovable. This is the quantitative analogue of the MIU modular-invariant argument: a preserved invariant proves impossibility, while a slowly changing potential proves length.

The hard part is not proving the inequality after a suitable r is found. The hard part is constructing a nontrivial rank that is preserved or slowly increased by every inference rule of a strong system and that assigns a large value to a formalization of $P \neq NP$.

5.4 The definitional floor

For a database Σ in which a target L is not yet expressible, define the definitional floor $D_\Sigma(L)$ as the least number of new logical assertions required before the target and all its constituent notions are both expressible and usable. Definitions alone are insufficient if no elimination, congruence, or manipulation lemmas accompany them.

Then

$$\|L\|_\Sigma \geq D_\Sigma(L).$$

For a suitable extension of the uploaded `set.mm`, the Predator document gives a calibration-based estimate

$$D_{\text{set.mm}}(P \neq NP) \sim 10^4$$

logical assertions and therefore suggests

$$\|P \neq NP\| \gtrsim 10^4$$

before the proof proper begins.

This is explicitly an estimate, not a theorem. The rigorous content is the abstract inequality together with a structural audit of what the database lacks. The constant and even the exact formalization boundary depend on how machine computation, polynomial time, nondeterminism, reductions, and complexity classes are developed.

One literal audit claim in the earlier document also requires updating. The uploaded current `set.mm` does contain comments using the word “Turing” and commentary mentioning computability theory. Those comments do not constitute a formal development of time-bounded machines or complexity classes, but they mean that substring counts are not an adequate audit method. The definitional floor should instead be justified by labels, dependencies, formal definitions, and usable inference lemmas.

5.5 The indispensable-dependency floor

Let $A(L)$ be a set of assertions that every fully expanded proof of L must use. Then

$$\|L\|_{F,\Gamma}^{\text{expanded}} \geq |A(L)|.$$

For $P \neq NP$, a meaningful version might try to prove the indispensability of modules for:

- deterministic and nondeterministic machine semantics;
- time-bounded computation;
- polynomial bounds and closure lemmas;
- reductions and NP-completeness;
- circuit or proof-system translations;
- at least one barrier-crossing ingredient.

The qualification “fully expanded” is essential. In a compiled library, a theorem with thousands of dependencies may be cited in one line. Conservative compilation changes proof length without changing the old-language theorem set. Therefore an indispensable-dependency result must specify whether it bounds inlined proofs, compressed proofs, or proof size under a designated proof system.

5.6 Restricted proof-system lower bounds

Strong lower bounds are known for restricted systems and explicit families. Resolution and bounded-depth Frege systems admit exponential or other substantial lower bounds on suitable tautology families. These results demonstrate that proof length can be intrinsically enormous in fixed systems.

They do not directly provide a lower bound on one ZFC or `set.mm` proof of the single sentence $P \neq NP$. The major gap is that no superpolynomial lower bound is known for unrestricted Frege proofs of any explicit tautology family, and Extended Frege is at least as difficult. A 2026 proof-complexity article still describes arbitrary-depth Frege lower bounds as a major open problem and Extended Frege lower bounds as still harder.²

Recent progress obtains strong bounds in more restricted settings, for example tree-like semantic Frege systems with bounded line size. Such advances are important but do not close the unrestricted Frege or strong-foundational-system gap.

5.7 Why the standard barriers do not yield line bounds

Relativization, natural-proofs, and algebrization barriers constrain the *content* of any successful proof. A proof of $P \neq NP$ must contain a non-relativizing ingredient, must evade the natural-proofs obstruction in the relevant sense, and must not merely algebrize in a way covered by the known barrier.

But a content restriction is not yet a length restriction. To infer

$$\|P \neq NP\|_{F,\Gamma} \geq N,$$

one would need to prove that every barrier-crossing ingredient costs at least N lines or symbols in F . No such general lower bound is known. The barriers exclude certain proof shapes; they do not count the lines in the remaining shapes.

5.8 What can honestly be claimed

For a strong system such as ZFC or a fully developed `set.mm` extension, there is presently no substantial unconditional numerical lower bound on the shortest proof of $P \neq NP$.

The most defensible forms are:

1. If $P \neq NP$ is false or independent of F , then

$$\|P \neq NP\|_{F,\Gamma} = \infty.$$

2. A rigorously computed definitional floor gives

$$\|P \neq NP\|_{F,\Gamma} \geq D_F(P \neq NP).$$

²Yaroslav Alekseev, “A Lower Bound for Polynomial Calculus with Extension Rule,” *Theory of Computing* 22 (2026), Article 4, <https://theoryofcomputing.org/articles/v022a004/index.html>.

3. A successful potential construction gives

$$\|P \neq NP\|_{F,\Gamma} \geq 1 + \left\lceil \frac{r(P \neq NP) - r_0}{d} \right\rceil.$$

4. A lower bound proved in a restricted proof system applies only after the relationship between that system and the intended foundational system has been specified.

The estimate of order 10^4 in the uploaded files is best treated as a formalization-cost planning estimate. It is not yet a proved lower bound on the mathematical argument that would settle P versus NP.

6 Conclusion

The five theorems isolate structural facts true throughout the ATP framework:

- every run is recoverable from formula birth times;
- every run is a chain even when the search space branches;
- finite formula universes impose a finite novelty budget;
- invertible recoding cannot create speedup;
- computable certified completion is impossible for undecidable consequence relations.

The three conjectures identify places where the framework meets genuine open proof-search questions: asymptotic value from the losing branch of settlement search, order-theoretic realization of settlement regions, and admissible learned heuristics with superpolynomial breadth savings.

ATPs may solve P versus NP, but only conditionally: a fair prover finds a finite proof if the proof exists in its system. Nothing guarantees that either side is provable in the chosen axioms, that the proof is short, or that a practical policy can locate it.

Finally, lower bounds on

$$\|P \neq NP\|_{F,\Gamma}$$

are potentially more decisive for mechanized search than arithmetical-tier classifications. Yet strong-system proof-length lower bounds are themselves among the hardest open problems in proof complexity. The best near-term program is therefore not to assert a large lower bound, but to develop auditable lower-bound mechanisms: definitional floors, indispensable dependency modules, slowly changing proof potentials, and explicit separations between proof systems.

Source note

The mathematical framework and source-derived claims in this document are based on the following uploaded materials:

- Brian Tenneson, *Depths of a Simulation and Formalized Self-Awareness*, Merged Edition, Version 1, July 27, 2026.

- Brian Tenneson, *Length, Not Height: On Proof-Length Lower Bounds, the MU Argument, and Whether a Self-Aware Theorem Prover Could Ever Settle a Clay Problem*.
- Brian Tenneson, *Predator 7.1 Documentation and Cross-Branch Reuse Rate Theory, with Efforts Toward Lower Bounds on the Settlement Length of P vs NP*, Revision 2, July 2026.
- Brian Tenneson, *ML-SIC-ATP Theory: Learned Search Policies for Proof Search*, Version 1.1, July 27, 2026.
- The uploaded `set.mm` database and accompanying prover source files.